# NP-completeness of fixed-order three-machine flexible blocking scheduling

Research manuscript — awaiting expert assessment
16 September 2026

## Abstract

Fixing the job order removes sequencing decisions from a flow shop, but flexible operation assignments may remain difficult. We give a deterministic polynomial-time reduction from Partition to makespan feasibility on three machines with blocking, nonpreemptive operations, and a fixed job order. Every job visits every machine; an item job has five operations, including one flexible operation at each adjacent machine pair. The construction preserves two cumulative weights, and a final job bounds their maximum. Every feasible target assignment yields a partition by reading one assignment bit per item. The result establishes NP-completeness for the general model permitting different operation patterns across jobs; it does not establish strong NP-hardness or hardness under a common operation pattern.

## 1. Introduction

A fixed production sequence leaves an assignment problem when an operation can be performed at either of two adjacent machines. With no intermediate storage, that choice also changes how long a job occupies an upstream machine. Complexity therefore depends on the interaction between assignment and blocking, even when job order is prescribed.

Nicosia et al. [1, Section 3.2] give a polynomial algorithm for the two-machine, fixed-order, nonpreemptive problem. Their Table 3 and conclusion leave the three-machine fixed-order case open. Section 2 defines the general operation model and then uses a common operation pattern to simplify presentation. We work with the general model: item jobs have five operations, while two boundary jobs have three. The identical-pattern restriction is outside our theorem.

**Theorem 1.** Makespan feasibility for fixed-order, three-machine, inter-stage flexible blocking flow shops is NP-complete, even when each job has at most five operations and at most one flexible operation per adjacent machine pair. Processing times are positive integers encoded in binary. There is a deterministic Karp reduction from positive-integer Partition, together with polynomial-time recovery of a partition from every feasible assignment of the constructed instance.

The construction maintains two partial weight sums in machine departure times. A large middle-machine duration excludes one unwanted assignment mode. A final job then enforces a bound on both sums, excluding a second unwanted mode and forcing equality. The proof gives weak NP-hardness through Partition [2]; it makes no claim of strong NP-hardness.

## 2. Model and earliest schedules

A source instance consists of positive integers $w_1, \dots, w_n$, with $n \geq 1$. It is in Partition if some subset has half the total weight. A source witness is its membership vector in $\{0,1\}^n$.

The target has three machines $M_1, M_2, M_3$ and a fixed list of jobs. Each job has an ordered operation list with one mandatory operation on each machine. An intervening flexible operation may use either adjacent machine, with its specified processing time. All times are positive integers. Assignments must be nondecreasing in machine index along a job. Operations are nonpreemptive, machines process jobs in the given order, and a job occupies its current machine until it starts on the next. The last machine releases it at completion. Simultaneous release and start are allowed. The question is whether completion of the last job can occur by an integer bound $H$.

For a fixed assignment, combine the consecutive operations on each machine into a processing triple $(a, b, c)$. Internal idle time cannot improve a schedule. Let $(d_1, d_2, d_3)$ be the preceding job's departure times, starting from zero before the first job. The next job's earliest departures are

$$e_1 = \max(d_1 + a, d_2), \quad e_2 = \max(e_1 + b, d_3), \quad e_3 = e_2 + c. \tag{1}$$

**Lemma 2.** The recurrence in Equation 1 computes the minimum makespan for a fixed assignment. Consequently, assignment lists are polynomial-size certificates for the target language.

**Proof.** The job can start on the first machine when its predecessor leaves. Its transfer to the second machine requires both its own processing to finish and the predecessor to depart that machine. The same argument applies to the third machine. These conditions give Equation 1 and a feasible schedule attaining it. Each update is monotone in previous departures. Induction therefore makes the computed departures no later than those of any feasible delayed schedule. Exact evaluation uses polynomially many integer operations on polynomial-length integers. □

## 3. The reduction rule

Let $W = \sum_{t=1}^{n} w_t$ and define

$$L = 4W + 4, \quad C = 6W + 6, \quad G = 5W + 5. \tag{2}$$

The prescribed job sequence is a primer with triple $(1, G, C)$, the $n$ item jobs in source order, and a suffix with triple $(G + C - 1, 1, 1)$. All primer and suffix operations are mandatory. Figure 1 shows an item job; the arrows specify operation precedence, not elapsed time.

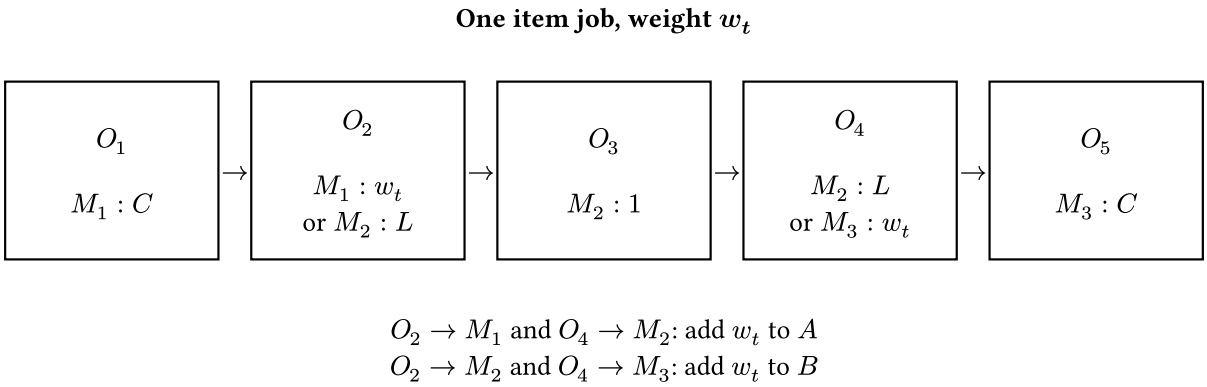



Figure 1: The five operations of the item with weight $w_t$. A label $M_k : p$ specifies processing time $p$ on machine $M_k$; "or" marks the two legal choices for a flexible operation. All arrows are precedence constraints. The two paired choices displayed below the job contribute to opposite cumulative sums. The proof excludes the other two combinations from every feasible assignment.

For an item, let $i = 1$ when its second operation uses $M_1$, and let $j = 1$ when its fourth operation uses $M_2$. These are local choice bits, not job indices. The four legal aggregate triples are shown in Table 1.

| Choice $(i, j)$ | $M_1$ | $M_2$ | $M_3$ |
|---|---|---|---|
| $(0, 0)$ | $C$ | $1 + L$ | $C + w_t$ |
| $(0, 1)$ | $C$ | $1 + 2L$ | $C$ |
| $(1, 0)$ | $C + w_t$ | $1$ | $C + w_t$ |
| $(1, 1)$ | $C + w_t$ | $1 + L$ | $C$ |

Table 1: Processing triples for all item assignments.

The forward algorithm emits these operation lists and the bound

$$H = 2 + G + (n + 1)C + \left\lfloor \frac{W}{2} \right\rfloor. \tag{3}$$

Invalid source encodings are mapped to a single mandatory job $(1, 1, 1)$ with bound zero, a legal NO instance.

The recovery algorithm reads the second-operation machine of every item job. It selects $w_t$ precisely when that operation uses $M_1$. Recovery is required for every valid target witness of the forward image. It does not assume that the witness minimizes the makespan or was produced by a particular solver.

## 4. Correctness

All departure variables below refer to the earliest schedule of Lemma 2.

### 4.1. An irreversible excess excludes one mode

Let $x_t$ denote the second-machine departure of item $t$, with $x_0 = 1 + G$ for the primer. Let $c_t$ be its third-machine processing time, and set $c_0 = C$.

**Lemma 3.** No assignment meeting the bound in Equation 3 contains mode $(0, 1)$.

**Proof.** The recurrence gives $x_t \geq x_{t-1} + c_{t-1}$. Define the nonnegative excess $\Delta_t = x_t - x_{t-1} - c_{t-1}$. For mode $(0, 1)$, the first-machine departure is at least $x_{t-1}$, so

$$\Delta_t \geq 1 + 2L - c_{t-1} \geq 1 + 2L - (C + W) = W + 3. \tag{4}$$

Every primer or item third-machine duration is at least $C$. Telescoping the increments, including this excess, shows that the last item departs the third machine no earlier than $1 + G + (n + 1)C + W + 3$. The suffix requires at least one further unit on that machine. This exceeds the bound in Equation 3, regardless of other item choices. □

### 4.2. Two sums survive blocking

For each remaining item mode, define $\alpha_t = w_t$ if its second operation uses $M_1$, and zero otherwise. Define $\beta_t = w_t$ if its fourth operation uses $M_3$, and zero otherwise. Its triple is $(C + \alpha_t, b_t, C + \beta_t)$, where $b_t \leq L + 1$. Put $\beta_0 = 0$.

**Lemma 4.** After item $t$, the first two departures are

$$d_{1,t} = 1 + tC + \sum_{h=1}^{t} \alpha_h, \quad d_{2,t} = 1 + G + tC + \sum_{h=1}^{t-1} \beta_h. \tag{5}$$

For $t = 0$, both sums are empty and the formulas denote the primer departures.

**Proof.** The primer establishes the base case. Each partial sum of the $\alpha$ values or the $\beta$ values lies between zero and $W$. The induction hypothesis therefore gives

$$G - W \leq d_{2,t-1} - d_{1,t-1} \leq G + W = C - 1. \tag{6}$$

The first maximum in Equation 1 selects $d_{1,t-1} + C + \alpha_t$. For the second maximum, the proposed value is $d_{2,t-1} + C + \beta_{t-1}$. Its difference from the new first departure is

$$G + \sum_{h=1}^{t-1} \beta_h - \sum_{h=1}^{t} \alpha_h \geq G - W = L + 1 \geq b_t. \tag{7}$$

Thus the preceding third departure determines the second maximum. This proves both expressions in Equation 5 without presupposing their active branches. □

Let $A = \sum_{t=1}^{n} \alpha_t$ and $B = \sum_{t=1}^{n} \beta_t$. The invariant and the last third-machine duration imply

$$d_{1,n} = 1 + nC + A, \quad d_{3,n} = 1 + G + (n + 1)C + B. \tag{8}$$

**Lemma 5.** If no item uses mode $(0, 1)$, the minimum makespan for the assignment is

$$2 + G + (n + 1)C + \max(A, B). \tag{9}$$

**Proof.** The suffix's second departure is

$$\max\bigl(d_{1,n} + G + C, d_{2,n} + 1, d_{3,n}\bigr). \tag{10}$$

Its middle term is redundant because $d_{3,n} = d_{2,n} + c_n$ and $c_n \geq 1$. Substitute the two displayed departures and add the suffix's final unit of processing to obtain Equation 9. □

### 4.3. Equivalence and extraction

**Proof of Theorem 1.** Given a partition, use mode $(1,1)$ for its selected weights and mode $(0,0)$ for all others. Then $A = B = \frac{W}{2}$, so Lemma 5 gives a legal schedule meeting the bound in Equation 3.

Conversely, take any feasible assignment. Lemma 3 excludes mode $(0,1)$. For the remaining choices, write $D$ for the sum of weights assigned mode $(1,0)$. The other two modes count a weight once, so

$$A + B = W + D, \quad A \le \left\lfloor \frac{W}{2} \right\rfloor, \quad B \le \left\lfloor \frac{W}{2} \right\rfloor. \tag{11}$$

If $W$ is odd, these inequalities are inconsistent. Otherwise, positivity of all weights forces $D = 0$ and $A = B = \frac{W}{2}$. Every item consequently uses $(1,1)$ or $(0,0)$. Selecting exactly those whose second operation is on $M_1$ recovers a partition. Lemma 2 supplies membership in NP, and Partition supplies NP-hardness. This proves both the Karp equivalence and recovery from every feasible target assignment. □

## 5. Encoding size and running time

Let $N$ be the source encoding length. Then $n \le N$ and $\log(W+1) = O(N)$. The construction has $5n + 6$ operations, durations $O(W)$, and bound $O(nW)$. Its binary output length is therefore $O(N^2)$. Schoolbook arithmetic and serialization give a conservative $O(N^3)$ bound for the deterministic forward algorithm. Validity checking and the fixed-NO extension are polynomial.

Recovery scans one assignment entry per item and outputs $n$ bits. With target witness length $Y$, parsing and extraction have a conservative $O((N+Y)^2)$ bit-time bound. Neither algorithm solves Partition, calls a solver, uses randomness, nor depends on state retained from construction. Decimal JSON is a transport representation of these integers; its length differs from binary by only a constant factor.

## 6. Scope and remaining questions

The result classifies the general fixed-order three-machine assignment problem under the stated blocking convention. It complements the two-machine polynomial case in [1]. The construction leaves open stronger restrictions, including a common operation pattern, and does not establish strong NP-hardness.

A dated search of primary literature on 16 September 2026 found no matching resolution within the inspected sources and equivalent formulations. That coverage does not include all unpublished or unindexed work. Correctness rests on the general proof; priority and significance require expert assessment.

## References

[1] Gaia Nicosia, Andrea Pacifici, Ulrich Pferschy, Anna Russo Russo, and Cecilia Salvatore. Flow shop scheduling with inter-stage flexibility and blocking constraints. *Computers & Operations Research* 184 (2025), 107219. doi:10.1016/j.cor.2025.107219. Preprint: arXiv:2411.18381 (2024).

[2] Richard M. Karp. Reducibility among combinatorial problems. In R. E. Miller and J. W. Thatcher (eds.), *Complexity of Computer Computations*, Plenum Press, 1972, pp. 85–103. doi:10.1007/978-1-4684-2001-2_9.

## A. Verification and reproducibility

Tests supplement the proof and do not prove correctness for all inputs. All commands below run from the campaign directory. The tested environment uses Python 3.14.7, Z3 5.1.0, and Typst. Z3 is used only by verification, not by the reduction or extractor.

The prepared suite executes actual forward outputs and obtains assignments with an independently implemented operation model. Its ten sources produced 14 valid recoveries. Separate exhaustive verification checks all positive weight lists of lengths one through three with values one through three: 39 sources, 1,884 assignments, and 24 recoveries. It also tests six malformed decoded objects, three malformed JSON strings, and two large-integer recovery cases. The current outputs are in `rounds/001/*after-repair.json`.

An independent reviewer implemented operation-start constraints without importing the candidate: seven selected sources yielded 628 assignments and ten recoveries. A focused transport check then verified six recoveries for 4,300-, 4,301-, and 6,001-digit integer cases, plus eight malformed forward inputs. These suites have overlapping domains and their counts are not additive.

```
python3 work/check.py --self-test
python3 work/check.py --candidate work/algorithm.py
python3 work/verify.py --candidate work/algorithm.py
python3 reviews/initial/check.py
python3 reviews/transport-repair/check.py
printf '%s\n' '{"weights":[1,1]}' | python3 work/algorithm.py
printf '%s\n' '{"source":{"weights":[1,1]},"target_solution":{"assignment":[[1,2,3],
[1,1,2,2,3],[1,2,2,3,3],[1,2,3]]}}' \
  | python3 work/algorithm.py --extract
typst compile work/figures/item.typ work/figures/item.svg
typst compile work/manuscript.typ work/manuscript.pdf
```

The recovery example returns `{"selected":[1,0]}`. Original review logs retain the pre-repair integer-transport failures; rerunning the initial checker on the current implementation reports the repaired behavior. All instance scopes are finite and no runtime timeout is imposed.